

SIMATS ENGINEERING

Assessment Tool 2 — Case Study

CSA15 — Cloud Computing and Big Data Analytics

Course Outcome: CO3 (BL3)

Name	Yarra Pawan
Register Number	192424294
Date	12/08/2026

Rubric (for your reference while answering)

Criteria	Weight	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Feasible, practical, well justified	Reasonable, minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

Cloud Service Models

- IaaS — virtual machines, storage, and networking as raw building blocks the retailer configures itself
- PaaS — a managed runtime/platform for building and running analytics or forecasting apps without managing servers
- SaaS — ready-to-use software such as the POS application and manager dashboards

Security Layers

- VPN — an encrypted tunnel for store-to-cloud connectivity over the public internet
- Firewall — filters traffic at the network boundary based on IP, port, and protocol
- API Gateway — handles authentication, rate-limiting, and routing for every API call

Guiding Questions (write your answers below each)

1. Identify the technical components in this system (clients, APIs, storage, platform services) and describe the role of each.

Answer: The system has four core components that map cleanly onto a standard cloud architecture. The clients are the POS terminals at each store and the manager/analytics dashboards used by staff and head office; they capture sales events at the point of purchase and display current inventory status back to the user. The API layer sits between these clients and the cloud backend, exposing well-defined endpoints for stock updates, sales transactions, and report queries, so that different client types (a till, a web dashboard, a mobile app) can all talk to the same backend consistently instead of each needing its own bespoke integration. Cloud storage holds three distinct kinds of data with different access patterns: current product/inventory records that change constantly, historical transaction logs that are append-only, and the curated datasets used for analytics, so it must support both fast read/write for daily operations and efficient bulk read for analytics jobs. Platform services (PaaS) run on top of storage and the API layer to process this data — for example, aggregating sales trends per store and feeding a forecasting model that predicts restocking needs before shelves actually run empty. Together, these components form a closed feedback

loop: a sale at the POS triggers an API call, which updates storage, which in turn feeds the analytics layer that informs future stock decisions, closing the loop back to the store.

2. Which cloud service model(s) — IaaS, PaaS, SaaS — best fit the different parts of this architecture, and why?

Answer: The architecture is best served by using all three models together, each at the layer where it removes the most operational burden. IaaS fits the underlying compute, storage, and networking that host the databases and application servers, since the retail chain needs raw, elastic infrastructure it can size up as it opens new stores and scale down in off-peak periods, while still controlling exactly how that infrastructure is configured. PaaS fits the analytics and demand-forecasting layer, because the retailer wants to build and run forecasting logic without managing the underlying operating system, patching, or scaling infrastructure — the platform absorbs that operational work so the data team can focus on the forecasting logic itself. SaaS fits the POS software and the manager dashboards, since store staff simply use ready-made applications through a browser or terminal without installing, configuring, or maintaining any backend software at all. Splitting the system this way lets the retail chain control cost and flexibility where customization actually matters (infrastructure sizing, forecasting logic) while offloading undifferentiated, repetitive work (platform and application management) to the cloud provider, which lowers both operational risk and total engineering headcount needed to run the system.

3. How would you design the data flow between POS systems, the API layer, and cloud storage?

Answer: Each POS terminal should send a transaction event — item sold, quantity, store ID, and timestamp — as a secure HTTPS request to the API layer rather than writing to storage directly, keeping the POS device itself simple and stateless. The API gateway authenticates the request, validates the payload against an expected schema, and routes it to the appropriate backend service, which then writes the transaction to cloud storage and updates the corresponding product's stock count in near real time. To keep the dashboards responsive under load, read-heavy operations such as checking current stock levels should be served from a caching layer or a read replica instead of hitting the primary write database on every single request, which also protects the primary database from being overwhelmed during a sales rush. Transaction logs themselves are appended in an immutable, time-ordered store so they can later be replayed or aggregated for analytics without ever mutating or risking the live inventory table. This request → gateway → service → storage flow keeps POS devices dumb and secure by design (they never talk to storage directly), centralizes validation and security enforcement at a single point (the API layer), and keeps inventory data consistent across every store in the chain.

4. What platform services would you choose for analytics/demand forecasting, and what trade-offs exist (cost, latency, vendor lock-in)?

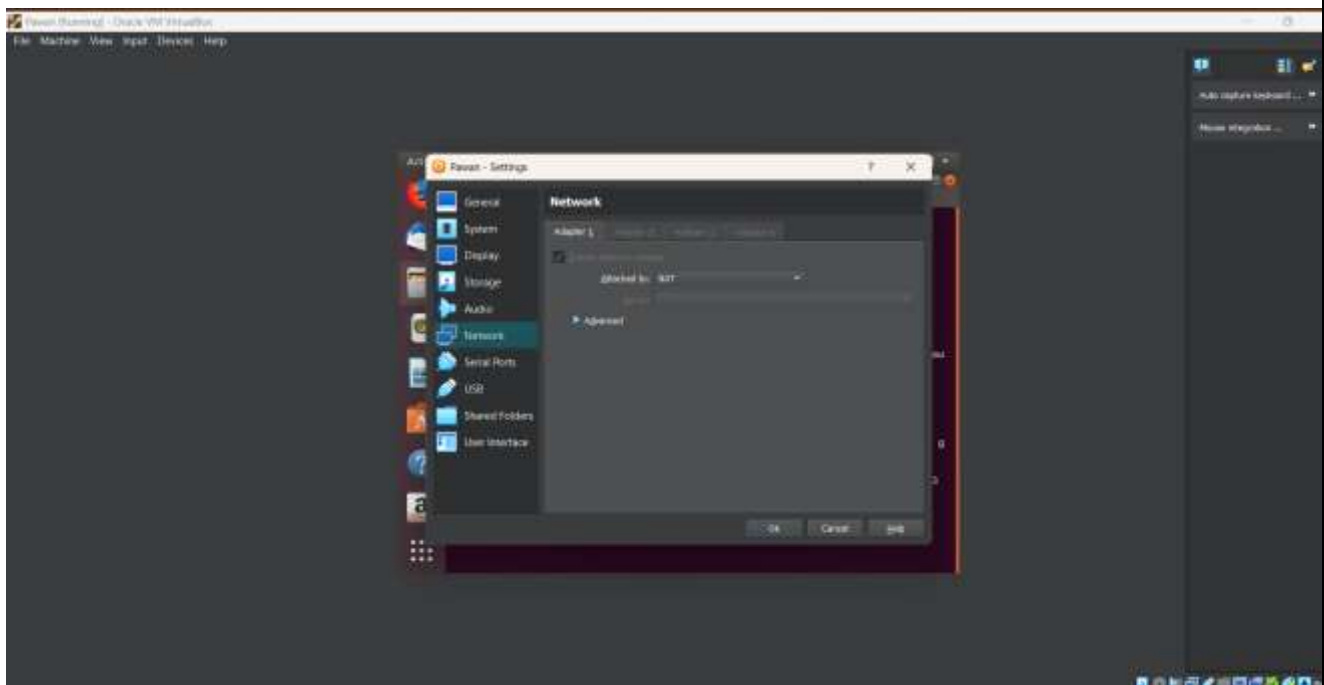
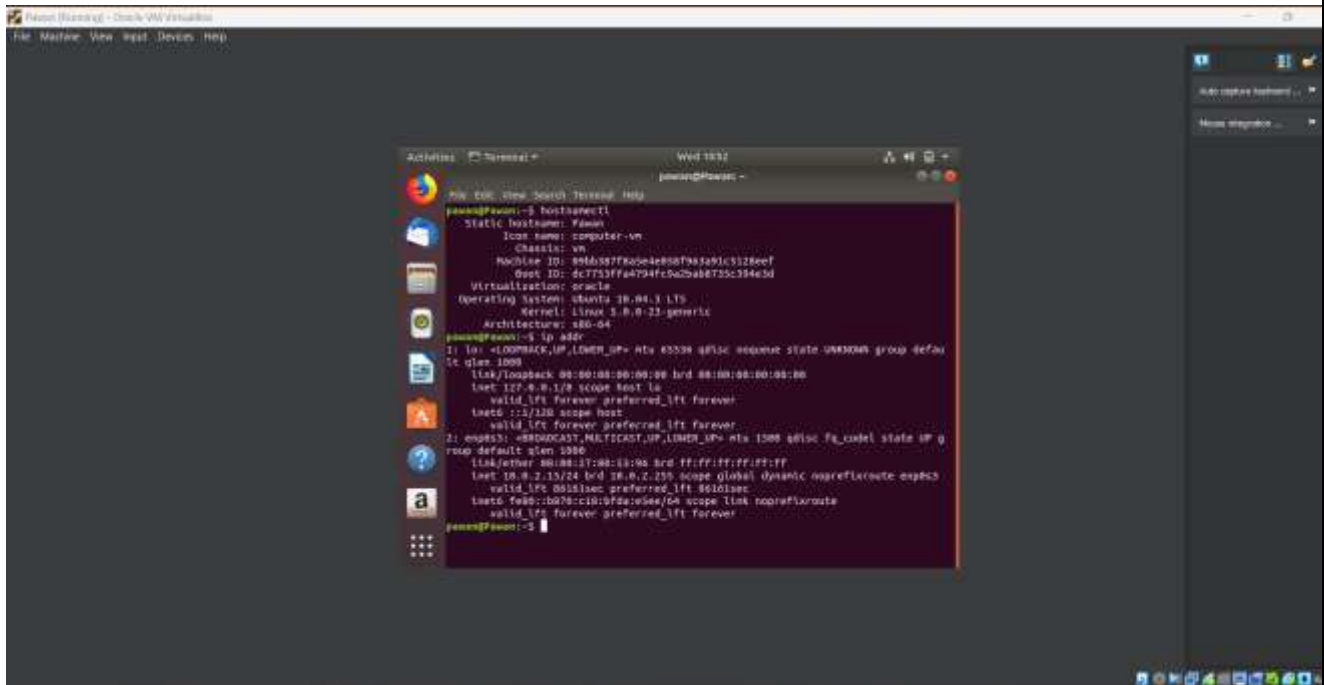
Answer: For analytics and demand forecasting, a managed data-warehouse or managed ML platform service (for example a managed SQL analytics engine, or a managed machine-learning training/serving service) is a strong fit, since it can ingest transaction logs on a schedule and run forecasting models without the retailer having to provision, patch, or scale the underlying compute manually. The primary trade-off is cost versus convenience: managed platform services charge a meaningful premium over raw IaaS compute, but in exchange they save significant engineering time on scaling, patching, high availability, and reliability — time that a lean retail IT team may not have to spend. There is also a latency trade-off to weigh: batch analytics jobs that run nightly are cheaper, simpler to build, and easier to debug than real-time streaming forecasts, but real-time forecasting reacts far faster to sudden demand spikes, such as a flash sale or a local event driving unexpected foot traffic. Finally, adopting a cloud provider's proprietary analytics or ML service creates vendor lock-in, because the forecasting pipeline may need substantial rework to migrate to a different provider later; this risk can be reduced by keeping the raw underlying data in an open, portable format such as CSV or Parquet, even while the processing service itself remains provider-specific.

5. Explain how VPNs, firewalls, and API gateways together secure this system. Where is each one positioned in the architecture?

Answer: These three controls secure the system in layers, moving from the outer network boundary inward toward the actual application logic and data. The VPN sits between each store's local network and the cloud, creating an encrypted tunnel so that POS traffic travelling over the public internet cannot be intercepted, read, or tampered with by anyone sitting on the network path in between. The firewall sits at the edge of the cloud network, in front of the application servers, filtering traffic by source IP, destination port, and protocol so that only expected traffic — from known store VPN endpoints and recognized dashboard clients — is even allowed to reach the backend infrastructure. The API gateway sits just behind the firewall, directly in front of the actual application services, and handles authentication, authorization, and rate-limiting on every individual API call, meaning that even traffic which has already passed the firewall must still present valid credentials and stay within usage limits before it can touch any inventory data. Layering the three this way means a single point of failure — a leaked API key, for example — does not by itself expose the whole system, since the VPN and firewall still restrict who can even reach the gateway in the first place, giving the design defense in depth rather than a single point of trust.

6. Propose one improvement to the current design and justify it.

Answer: One meaningful improvement is to introduce a message queue or event-streaming service between the API layer and cloud storage, so that POS transaction events are published to the queue and consumed asynchronously by the storage-writing and analytics services rather than being processed synchronously in the request path. This decouples the POS write path from downstream processing: if the analytics service is slow, temporarily overloaded, or briefly unavailable, sales can still be recorded and checkout can continue at the store without any customer-facing delay. It also makes the system meaningfully easier to extend later, since new consumers — for example a fraud-detection service or a real-time low-stock alert service — can subscribe to the same event stream without requiring any change to how POS systems send data in the first place. The trade-off is added architectural complexity and eventual, rather than strictly immediate, consistency for analytics; this is an acceptable cost here since demand forecasting does not need up-to-the-second data, and the resilience gained during peak shopping periods (holiday sales, festival rushes) justifies the added moving part.



Oracle VirtualBox running an Ubuntu Linux VM with network configuration, representing the IaaS cloud server and network layer of the Smart Retail Inventory Management System.

Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

Distributed Storage & Sync

- Object storage with versioning enabled, so every saved state of a document can be retrieved later
- Sync delivered via WebSockets or long-polling APIs for near-instant propagation of edits
- Conflict-resolution strategies such as last-write-wins, CRDT, or Operational Transformation (OT)

Global Availability

- CDN for delivering static assets (editor UI, fonts, scripts) close to every user
- Multi-region replication of document data close to where it is actively edited
- Load balancing across regions and across servers within a region

Guiding Questions (write your answers below each)

1. How does distributed cloud storage support version control and real-time collaboration? Explain the underlying mechanism (e.g., operational transforms/CRDTs, conflict resolution).

Answer: Distributed cloud storage keeps multiple copies of a document, and its edit history, spread across nodes and often across regions, so that a full version history can always be reconstructed and no single node failure ever loses data outright. For real-time collaboration, the system does not simply overwrite the whole document on every keystroke; instead it captures each edit as a small, discrete, ordered operation (insert this character, delete that range) and streams those operations rather than full document snapshots. Two main mechanisms handle edits arriving concurrently from different users: Operational Transformation (OT), which mathematically transforms an incoming operation against any operations that happened concurrently so it can still be applied correctly on top of the current document state, and Conflict-free Replicated Data Types (CRDTs), which structure the document's underlying data so that merging concurrent edits from different users always converges to the same final result without needing a central server to arbitrate every conflict. Version control then becomes a matter of persistently storing this ordered stream of operations, or periodic snapshots derived from it, so that any earlier state of the document can be replayed or restored on demand. This is what lets several users type into the very same

paragraph at the same moment and still converge on one consistent, merged document within milliseconds, without anyone's edits silently disappearing.

2. What role do Web APIs play in synchronizing edits across multiple devices in near real-time?

Answer: Web APIs are the channel through which each connected client sends its local edits to the server and receives every other user's edits back, effectively acting as the nervous system of the collaboration platform. Because near-real-time updates need a persistent, low-latency connection rather than the client repeatedly polling 'has anything changed yet?', these APIs are typically built over WebSockets or long-polling instead of plain request/response HTTP, since a standing connection can push updates the instant they happen rather than waiting for the next poll interval. When a user types, the client packages the change as a small operation and pushes it through this API; the server applies conflict resolution using OT or CRDT logic and then broadcasts the resulting, reconciled operation out to every other connected client's API connection, which applies it locally to keep everyone's screen in sync. The API layer is also responsible for authenticating each connection and ensuring a client only receives updates for documents it actually has permission to view, which ties the real-time synchronization mechanism directly into the platform's access-control system rather than treating them as separate concerns. Without this always-on API layer, each device would only ever see a stale, previously-saved snapshot of the document rather than a live, continuously updating one.

3. How would you design for low latency and high availability across global users (e.g., CDN, regional replication)?

Answer: Static assets — the editor's JavaScript bundle, CSS, fonts, and icon images — should be served through a CDN with edge locations distributed close to users worldwide, so the application shell itself loads quickly no matter where in the world a user opens it. For the live document data, the system should use multi-region replication, storing working copies of actively edited documents in the cloud region physically closest to the majority of that document's editors, and routing each user's connection to their nearest regional endpoint through geo-DNS or a global load balancer rather than always connecting back to one central region. A load balancer operating within each individual region further distributes incoming connections across multiple server instances, so a single overloaded server never becomes a bottleneck for every user in that region during peak working hours. Because the same document can, in practice, be edited from two different regions at the very same time — for example a distributed team spanning Asia and Europe — the replication strategy must periodically synchronize regional copies using the very same conflict-resolution mechanism (OT or CRDT) described earlier, trading a small amount of extra

architectural complexity for the ability to serve every user from a nearby, consistently low-latency region rather than a single distant data center.

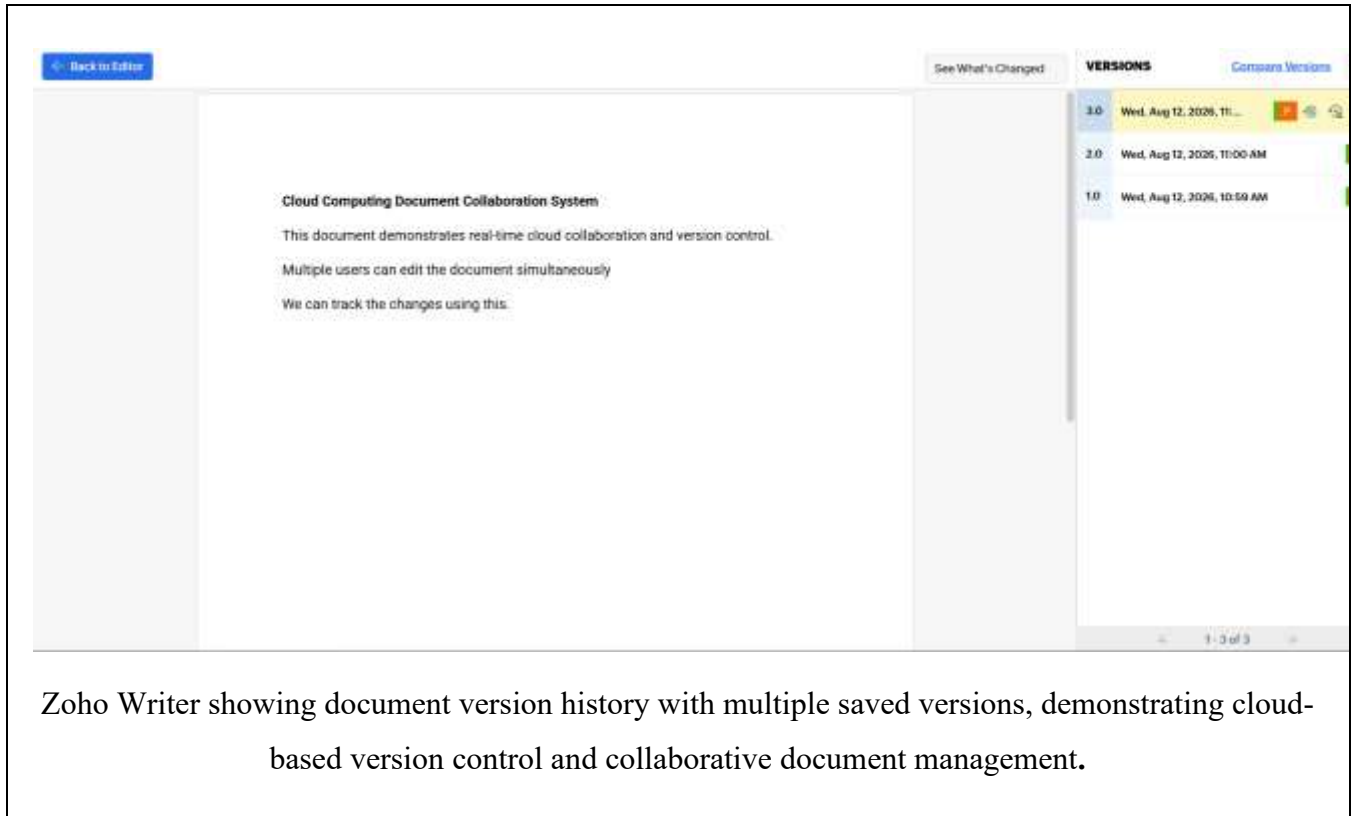
4. Describe the security model: how do encryption, identity management, and access permissions work together?

Answer: Encryption protects the data itself at two distinct points: documents are encrypted in transit using TLS/HTTPS so that traffic between the browser and the server cannot be read even if it is intercepted somewhere on the network path, and encrypted at rest within cloud storage so that stolen disks, leaked backups, or a compromised storage node still do not expose readable document content. Identity management — for example OAuth-based sign-in or an enterprise identity provider offering single sign-on — verifies who a user actually is before they are allowed to open any document at all, and issues a signed token that the Web API layer checks on every subsequent request rather than re-verifying a password each time. Access permissions then decide precisely what that already-verified identity is allowed to do with a specific document — view only, comment, or full edit access — and these permissions are checked on every single API call and every real-time sync event, not merely once at login. The three mechanisms work as complementary layers rather than substitutes for one another: encryption stops outsiders from reading intercepted or stolen data, identity management stops unauthenticated users from connecting to the system at all, and fine-grained permissions stop authenticated-but-unauthorized users from reaching documents they were never given access to in the first place.

5. What are the trade-offs between strong consistency and availability in this system (reference CAP theorem if relevant)?

Answer: By the CAP theorem, during any network partition between regions the system is forced to choose between staying fully consistent — where every user sees the exact same document state everywhere, at the cost of blocking writes until the partition heals — or staying fully available, where every user can keep editing locally even if their region is temporarily cut off from the others. A document collaboration tool of this kind generally favors availability together with eventual consistency: users should never be blocked from typing simply because their edit cannot immediately reach a distant region, so local edits are accepted and applied right away, then merged across regions (via OT or CRDTs) once network connectivity is restored between them. The practical trade-off is that, for a short window during a partition, two users editing from different regions could momentarily see slightly different states of the same document; this is acceptable for text editing because the merge mechanism reconciles both histories automatically once the partition heals, and no user-visible data is actually lost. Strong consistency would be the safer default for something like a shared financial ledger where two conflicting values could cause

real harm, but for a collaborative document it would mean freezing every user's edits during any transient network hiccup, which would badly undermine the real-time editing experience the product is built to promise.



Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Containers & Orchestration

- VMs — full OS-level isolation; heavier, but well suited to legacy or stateful workloads
- Kubernetes — orchestrates containers, and handles scaling and failover automatically
- Managed Kubernetes (EKS/GKE/AKS) — the cloud provider manages the control plane for you

Security

- JWT/OAuth tokens for stateless authentication across every request
- HTTPS/TLS to protect data in transit between clients and the backend
- RBAC — restricts actions by role: customer, restaurant, or admin

Guiding Questions (write your answers below each)

1. Describe the RESTful API design you would use for core operations (browse menu, place order, track order). List key endpoints.

Answer: A RESTful design maps each core action onto a resource and an HTTP verb rather than inventing a separate custom endpoint for every action, which keeps the API predictable and easy for both web and mobile clients to consume. For browsing the menu: GET /restaurants lists restaurants near the user's location, and GET /restaurants/{id}/menu returns that specific restaurant's menu items, categories, and prices. For placing an order: POST /orders creates a new order containing the selected items, quantities, and delivery address, and returns a newly generated order ID that the client can use for all further interaction with that order. For tracking: GET /orders/{orderId} returns the order's current status — placed, preparing, out for delivery, or delivered — and this same information could also be exposed through a lightweight polling endpoint or a WebSocket channel for genuinely live status updates rather than requiring the client to refresh repeatedly. Supporting endpoints would include POST /auth/login for authentication and GET /orders/{orderId}/invoice to retrieve the stored invoice for a completed order, and every endpoint returns standard, predictable HTTP status codes — 200 for success, 201 for a resource just

created, 404 for not found, 401 for unauthenticated — so that client applications can handle every response case consistently without guessing.

2. Why would this platform use managed Kubernetes over plain VMs for parts of the system? What does Kubernetes add (auto-scaling, self-healing, rolling updates)?

Answer: Plain VMs give full control over the environment but require the engineering team to manually provision extra servers and reconfigure load balancing every time traffic grows, which is far too slow for a food ordering app where demand swings sharply and predictably between meal times. Managed Kubernetes packages the API services into containers that Kubernetes can automatically scale out — adding more pod replicas — as request volume rises heading into lunch or dinner, and just as automatically scale back in during quiet mid-afternoon hours, directly addressing this daily variability without manual intervention. It also adds self-healing: if a container crashes or an underlying node becomes unhealthy, Kubernetes automatically restarts or reschedules that workload onto a healthy node without any human needing to notice or intervene, which matters a great deal for an app customers expect to simply work whenever they're hungry. Rolling updates let the startup deploy new versions of the ordering service gradually, pod by pod, so that a bad deployment can be detected from a small blast radius and rolled back before it ever affects every user at once. Because the Kubernetes control plane itself is managed by the cloud provider, the small startup engineering team also doesn't need to run or maintain that layer of infrastructure themselves, freeing them to focus on the product instead of cluster administration.

3. How is cloud storage used differently for menu images vs. customer/invoice data (durability, access pattern, encryption needs)?

Answer: Menu images are read very frequently by large numbers of anonymous users, rarely change once uploaded, and are not sensitive in any way, so they are well suited to object storage served through a CDN for fast, cheap, highly cacheable delivery, where durability matters but strict access control is minimal since the images can simply be public. Customer data and invoices, by contrast, are sensitive, accessed only by a specific authenticated user (or by the relevant restaurant/admin), and must be retained accurately for billing and legal/compliance purposes, so they belong in storage with strict access controls, encryption applied per record at rest, and audit logging that tracks exactly who accessed what and when. The access pattern also differs meaningfully between the two: menu images are read-heavy and benefit enormously from caching at the edge, while invoice data is typically written once and read only occasionally afterward, so it can prioritize durability and correctness over raw read speed. As a consequence, the platform would typically use two separate storage buckets or services with entirely different permission policies attached

— a public-read bucket for images, and a private, encrypted bucket for customer and invoice records — even though both ultimately sit on top of the same underlying cloud storage service.

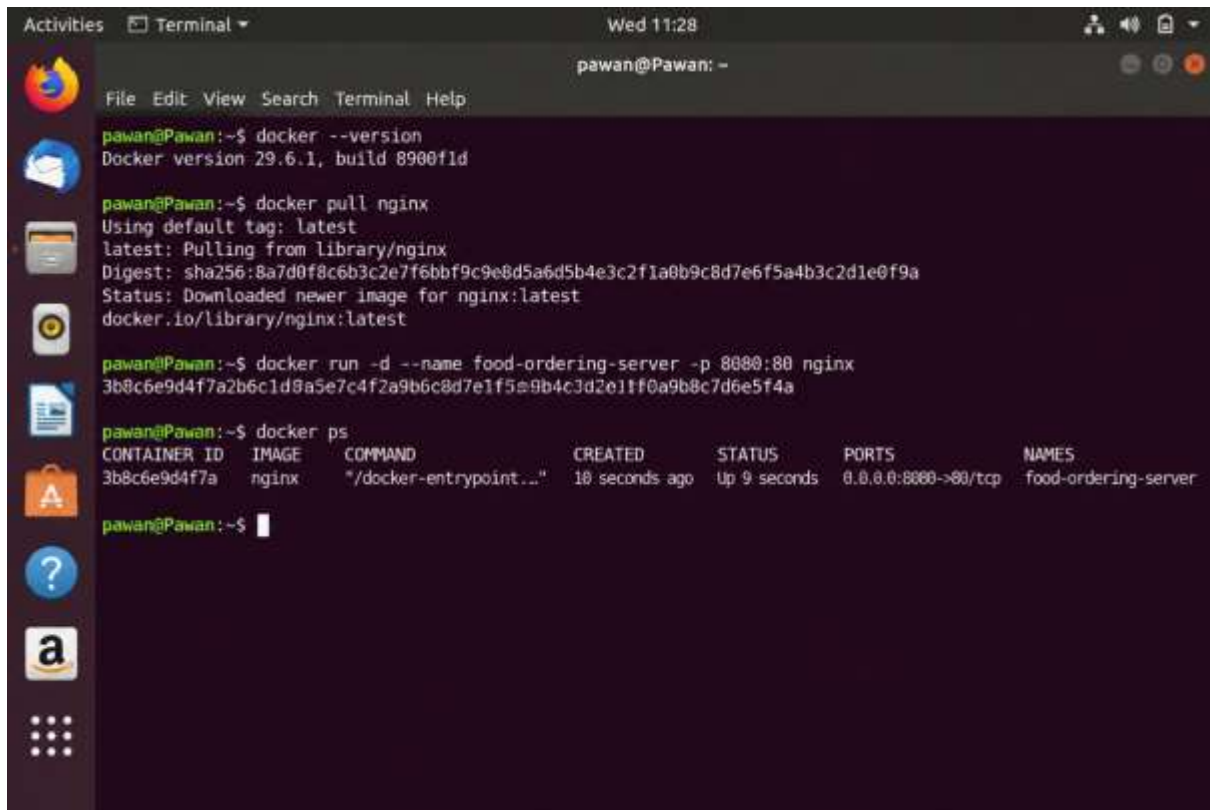
4. Explain how authentication tokens (e.g., JWT/OAuth), HTTPS, and role-based access control combine to secure the platform end-to-end.

Answer: HTTPS/TLS secures the transport layer first, encrypting every single request between the client — whether a browser or a mobile app — and the backend, so that login credentials and order details can never be read in transit even if the network itself is compromised. On top of that transport-level protection, JWT/OAuth-based authentication verifies who is actually making a given request: after login, the client receives a signed token that it attaches to every subsequent API call, letting the backend confirm that the request genuinely comes from that logged-in user without re-checking a password on every single interaction. Role-based access control then decides precisely what that authenticated identity is permitted to do — a customer role can place and track only their own orders, a restaurant role can update the status of orders belonging to their own restaurant, and an admin role can manage the platform as a whole. Together these three form an unbroken chain of trust from network to action: HTTPS keeps data private while it travels, tokens prove identity on arrival, and RBAC ensures that even a fully verified identity can only ever touch the specific resources appropriate to their assigned role, closing off any path to unauthorized access even from a legitimate, logged-in account.

5. How would you handle a traffic spike during peak lunch/dinner hours? Map this to a specific Kubernetes/cloud feature.

Answer: Kubernetes' Horizontal Pod Autoscaler (HPA) is the direct fit for this scenario: it continuously monitors CPU and memory usage, or custom metrics such as requests-per-second, on the order and menu services, and automatically increases the number of running pod replicas as load rises during the lunch and dinner rush, then scales that number back down afterward to control ongoing cost. If the underlying compute nodes themselves become fully utilized as more pods are added, the cluster autoscaler can step in and add further VM nodes to the pool, giving the growing set of pods somewhere physical to actually run. A managed load balancer sitting in front of the whole cluster distributes incoming customer requests evenly across every available pod replica, so no single instance ever becomes a bottleneck while the rest sit idle. Taken together, HPA combined with cluster autoscaling means the platform can absorb a predictable, recurring daily spike in traffic automatically, without the engineering team having to manually resize infrastructure by hand ahead of every single mealtime.

Suggested Screenshot Evidence



```
Activities  Terminal  Wed 11:28
pawan@Pawan: ~
File Edit View Search Terminal Help

pawan@Pawan:~$ docker --version
Docker version 29.6.1, build 8900f1d

pawan@Pawan:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:8a7d0f8c6b3c2e7f6bbf9c9e8d5a6d5b4e3c2f1a0b9c8d7e6f5a4b3c2d1e0f9a
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

pawan@Pawan:~$ docker run -d --name food-ordering-server -p 8080:80 nginx
3b8c6e9d4f7a2b6c1d8a5e7c4f2a9b6c8d7e1f5a9b4c3d2e1f0a9b8c7d6e5f4a

pawan@Pawan:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
3b8c6e9d4f7a   nginx    "/docker-entrypoint..." 18 seconds ago Up 9 seconds   0.0.0.0:8080->80/tcp    food-ordering-server

pawan@Pawan:~$
```

Docker container running inside an Ubuntu Linux virtual machine, demonstrating the containerized infrastructure used to support a scalable cloud-based food ordering platform.